FRED TALKS

12th November 2026

CONTENTS

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 1431 WORDS

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 3941 WORDS

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 945 WORDS

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 30 MAR 2026 · SOURCE

People code switch. Every ethnic minority has lived experience of this. When you talk to me in Singapore, I sound very different than when I'm in San Francisco. And the way you talk at work is probably very different from how you talk to your kids and how you talk to your lover.

I've been dealing with a particularly virulent form of code switching in the simple form of titling blogposts and talks. When you ask the writer face to face what their thing is about, they can give you a perfectly human explanation. And then when they send you their piece, it's all "***Leveraging GenAI with Robust Data Foundations to Transform Businesses***". I've left my body by the time I read the first word.

My hope in writing this piece is to **put an end to bad titles**.

If I sent you this guide, it is not an insult. You just don't have good title game. That's ok. You're great at what you do, title game isn't your job. I'm the guy that wrote How To Name Things and Two Words. I sent you this guide ***because*** your shitty title is the main thing stopping your readers from getting to the good part. If you didn't have anything good, I wouldn't bother with a lost cause.

First…

Towards Leveraging Corporate Speak For Maximum Enterprise Synergy

DO NOT:

- Start with "Leveraging" or "Towards" or "Navigating" or "Empowering"

Yeah just stop. There's no rescuing this when you start there. Just delete meaningless corporate weasel words. Anything >3 syllables is a red flag.

Make It Quotable

If you do not try to make your title roll off the tongue, it probably won't.

When working with a cache, I will often spell out in my prompt that I don't mind the data sometimes being stale or briefly inconsistent between cache instances. When reviewing Golang code, the first thing I do is scan for added mutexes or layers of indirection that don't look necessary to me.

If you allow requirements like these to propagate unchecked through a codebase, they will petrify the system over time. The whole thing will be such a mess of checking and re-checking and holding locks and enforcing consistency that it cannot be scaled at all. Keeping systems elegant requires a ruthless focus: on accepting stale data or occasional tail latency, on avoiding all but critical instances of coordination, and in general on figuring out what requirements you *don't* have to satisfy.

In most cases, an engineer who understands the technical side must try and explain it to a product manager, who can make a judgement on the business side. But this is a very difficult partnership to get right. Engineers and PMs have different incentives and little ability to check each other's work, which makes it hard to build trust. Without trust, the product manager is liable to avoid giving any ground at all, which leaves the design in an impossible state.

This is an underappreciated reason why solo developers or startups can sometimes run rings around larger companies. Because the technical and business decisions are united in a single person, they can make design tradeoffs that would be impossible for a large company, enabling them to build systems that are far more elegant and powerful.

Working with LLMs

[SVG OMITTED]

Knowing what requirements you don't have becomes far more important when working with coding agents. Frontier models are very capable, but they have all gone through the purifying flames of RLVR: like high-achieving schoolchildren, they are obsessed with satisfying the Grader. On top of that, they have very little business context compared to an experienced human. That often manifests as them catching correctness or consistency "bugs" that are really just hypothetical unsatisfied requirements.

For instance, it's not uncommon for an LLM to casually introduce a mutex or other concurrency mechanism to ensure that two pieces of semi-related data are kept in sync, even if not told to do so. I've seen LLMs add weird checks in the middle of request handling to ensure that some already-checked piece of data hasn't changed mid-flight, even if that wouldn't successfully prevent the feared TOCTOU bug. LLMs are obsessed with edge cases like "what if the current user loses their admin status in the middle of us handling one of their admin requests?", or "what if the cache in storage partition A is updated a nanosecond before the one in storage partition B, leading some users to see different data?". Particularly if run in a loop or told to "make no mistakes", they will pile on cases like these until they've transformed your four-line change into a thousand-line monster.

This is why I'm confused by some people's claims that code review is dead. About half the time I review a large AI-generated change, I notice some unnecessary requirement that makes the diff more complex. When I dig deeper, the LLM invariably introduced this requirement on its own.

Literally imagine telling your friend about your talk. Telling a new acquaintance to google your blogpost. Having a fan of yours cite it back to you. How much do they stumble? Yeah. This has all happened to me.

There's an extremely high correlation between what are considered the best talks and the quotability of titles. Simple Made Easy. Inventing on Principle. Our list of best demos/pitches of all time.

Resist colons. It lacks confidence. Why Use Lot Word: ~~When Few Word Do Trick~~? You're not High School Musical: The Musical: The Series.

Put the SEO keyword vomit in the description.

Titles are for humans.

Prescribe A Strong Opinion

Opinions are one of the most quotable things.

But more importantly, a good Opinion Piece is load bearing because it is eminently forwardable, and therefore viral and impactful. Your Job To Be Done is to be the definitive, conclusive, piece on that Opinion. Stake out territory in Opinion Space, list out all the arguments for, counter all the arguments against, provide datapoints, and watch that go further than anything you've ever done.

How to know what's a good opinion? **Three Strikes rule**: If you repeat yourself three times, ship it. Even if you're just quoting others, its fine, credit them and make it your own.

Mental Models and Frameworks are more complex opinions that allow others to scaffold their thought around you. My Radiating Circles and Measuring Developer Relations and Third Age of JavaScript did this.

Opinions contrary to an established opinion are the hardest to do well and the best when correct. What Clayton Christensen Got Wrong made Ben Thompson's career. Rise of AI Engineer countered the prevailing Prompt Engineer -and- GPU Rich Startup/ML Engineer narrative at the time, with room to grow.

Curiosity Gap Dynamics

Marketers know the power of the Curiosity Gap. Here is where we toy closest with clickbait. I'll be honest that I wasn't even sure if I wanted to give it its own subheading, but its a very important concept that you should at least be aware exists, even if you're going to ignore it.

Strong enough opinions create their own curiosity gaps; you won't have to try very hard to make it interesting if you got my attention in 4 words anyway.

Curiosity gaps done poorly can lead to "burying the lede", which make your talk less Quotable and you also don't want to do that.

Simplest way to put it is; if the most interesting thing about your content is your title, and what people take away after having read your post matches what they would have predicted after reading your title, then you've probably said too much and could cut.

Play Madlibs

You don't have to invent your title from scratch (nor should you always do what I'm about to suggest). Here's some templates that work well:

- {*BENEFIT EVERYONE WANTS*} **without** {*DOWNSIDE EVERYBODY EXPECTS*}
  - e.g. How to Get Rich (Without Getting Lucky), How to Market Yourself (Without Being A Celebrity). See also subtitle of this post.
- {*PRESCRIPTIVE OPINION*}, **don't** {*THING YOU DO*}
  - e.g. Parse, don't validate
  - alternative prescriptive opinion: **Never** {THING YOU DO}
  - see also title of this post
- {*POPULAR THING*} **IS DEAD**
  - unfortunately this is clickbait that works
  - e.g. Prompt Engineering is Dead, Big Data is Dead
  - when given at a conference for *POPULAR THING*, this is known as a Heresy Talk

Elegant systems drop some requirements

[SVG OMITTED]

For instance, take preforking web servers like Unicorn. Because they lack the feature of shared mutable state between server instances, they can run each server in its own process, which provides isolation for free. Because Unicorn specifically ignores the problem of slow client handling, it can queue requests directly on Unix sockets, drastically simplifying the implementation.

Because S3 does not support atomic multi-key updates — you cannot update multiple blobs in a single transaction — it can store data at the blob level however it chooses. That makes it much faster, since reads and writes don't have to cross storage partitions, and much easier to scale, since less coordination means fewer sources of truth.

StatsD uses UDP by default, which means it doesn't guarantee that every piece of data you send makes it into storage. In exchange for dropping that requirement, it incurs much less network overhead, which is a big advantage for a stats aggregation service that's constantly shoveling data over the wire.

The general design approach of fanning out data to an edge cache buys a lot of speed and scaling by dropping the requirement that data must always be up-to-date. SQLite buys operational simplicity — it can run out of a single file! — by dropping the requirement of concurrent writes. By accepting high tail latency, Cursor can scale their distributed git backend more easily. Well-designed systems turn their weaknesses into strengths.

Why dropping requirements is hard

[SVG OMITTED]

Anyone who's spent time designing real-world software (or who's studied distributed systems) knows that system design is a process of compromise. If you attempt to satisfy every single potential requirement — speed, concurrency, up-to-date data, a simple interface, supporting complex user behaviors — you will fail to design a working system at all.

Figuring out what requirements you can relax is difficult because **it combines both business and technical expertise**. It is technically challenging to identify precisely the level of staleness that a particular design change will allow; it is likewise challenging from a business perspective to determine whether that level of staleness is acceptable to users.

Wrap-up#

Eight years is a long time to carry a project in your head. Seeing these SQLite tools actually exist and function after only three months of work is a massive win, and I'm fully aware they wouldn't be here without AI.

But the process wasn't the clean, linear success story people usually post. I lost an entire month to vibe-coding. I fell into the trap of managing a codebase I didn't actually understand, and I paid for that with a total rewrite.

The takeaway for me is simple: AI is an incredible force multiplier for implementation, but it's a dangerous substitute for design. It's brilliant at giving you the right answer to a specific technical question, but it has no sense of history, taste, or how a human will actually feel using your API. If you rely on it for the "soul" of your software, you'll just end up hitting a wall faster than you ever have before.

What I'd like to see more of from others is exactly what I've tried to do here: honest, detailed accounts of building real software with these tools; not weekend toys or one-off scripts but the kind of software that has to survive contact with users, bug reports, and your own changing mind.

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 19 OCT 2026 · SOURCE

When designing software, the requirements you don't have are as important as the requirements you do. Identifying what you can compromise on — what can be slow, or stale, or inflexible — is usually the key to an elegant design.

    - opposite also works:
        - **The Rise of** {THING}
        - **The Unreasonable Effectiveness of** {THING}
- **Fundamentals of** {THING}
    - straightforward, people love fundamentals
    - also **How** {THING} **Really Works**
    - You can also flip it and go for the **Advanced** {THING} tier content
- {CONVEY EXPERIENCE/SCALE} with {THING}
    - e.g. What We Learned from a Year of Building with LLMs, Insights from over ***10,000*** comments on "Ask HN: Who Is Hiring" using GPT-4o

There's loads more templates that work, please suggest in comments and I can add/review.

You can also browse the masters of microcopy like Steph Smith and Shaan Puri, and if you are worried about clickbait, you can see Veritasium's take or Simon Willison's Highlights.

also Andrew Chen senpai:

Understand Buzzword Metagame

Buzzwords work. Because of a mix of human psychology and algorithmic recsys. Don't avoid them, just use them judiciously. This involves the ability to read the room, which many smart people sadly cannot without significant involvement in the broader community you are trying to speak to. I don't see any way around it other than trying to authentically immerse, or consult a friendly who does.

Not all buzzwords are created equal. Databricks/Berkeley tried to push Compound AI Systems, Gartner is pushing Composite AI and Composable business — this resonates with their audience, but not more broadly.

Pick the shibboleths that work for the community you want. AI Engineer worked for reasons that are well understood now. GraphRAG is working outside of the company that coined it. These buzzwords travel, go ahead and put them in there.

Buzzwords rise and fall. Try to catch them on the rise — go harder than everyone else on the rise up and regularly test their effectiveness, such that by the time your main buzzword has tapped out you've got 2-3 others cooking.

Workshopping your titles

I write this post out of frustration, but also to capture my own thought process. I'm currently procrastinating while writing a Latent Space post.

- The original title was Why every AI Engineer needs an AI Gateway. Good, genuine opinion, load bearing. But not super memorable.
- My next iteration was "**Stop writing model routers, use an LLM Gateway**". Ok, prescriptive, evocative. But didn't cover the other elements other than model routing. One could shorten to "Stop writing your own LLM Gateway" but that doesnt quite express what people do.
- My next one is "**It's not your job to DIY an LLM Gateway**". No buzzword bc "your job" is implied (for a Latent Space audience), a little curiosity gap, LLM buzzword, strong opinion. But still not strong enough.
- Final one is "**LLM Gateway: The One Decision That Removes 100 AI Engineering Decisions**". Leans on Tim Ferriss' madlbs.

It's very common for the "tail to wag the dog to wag the tail" - the title is so important to clickthrough and so impactful to the message, that you spend a bunch of time on just the title itself, because that determines the piece you will write, then write the piece, then you go back and tweak the title based on what happened.

Amateur writers think their content has 95% of the value and then spend 5% on the title. Pros will put the value of titles closer to 50% and then act like it.

You Should Break The Rules Once You Understand Human Behavior More Intuitively Than Rules Can Model

See title of this post. No better aura than intentional, conspicuous, successful rulebreaking.

Relativity#

Reflecting on the above, the pattern of when AI helped and when it hurt was fairly consistent.

When I was working on something I already understood deeply, AI was excellent. I could review its output instantly, catch mistakes before they landed and move at a pace I'd never have managed alone. The parser rule generation is the clearest example[37]: I knew exactly what each rule should produce, so I could review AI's output within a minute or two and iterate fast.

When I was working on something I could describe but didn't yet know, AI was good but required more care. Learning Wadler-Lindig for the formatter was like this: I could articulate what I wanted, evaluate whether the output was heading in the right direction, and learn from what AI explained. But I had to stay engaged and couldn't just accept what it gave me.

When I was working on something where I didn't even know what I wanted, AI was somewhere between unhelpful and harmful. The architecture of the project was the clearest case: I spent weeks in the early days following AI down dead ends, exploring designs that felt productive in the moment but collapsed under scrutiny. In hindsight, I have to wonder if it would have been faster just thinking it through without AI in the loop at all.

But expertise alone isn't enough. Even when I understood a problem deeply, AI still struggled if the task had no objectively checkable answer[38]. Implementation has a right answer, at least at a local level: the code compiles, the tests pass, the output matches what you asked for. Design doesn't. We're still arguing about OOP decades after it first took off.

Concretely, I found that designing the public API of syntaqlite was where this hit home the hardest. I spent several days in early March doing nothing but API refactoring, manually fixing things any experienced engineer would have instinctively avoided but AI made a total mess of. There's no test or objective metric for "is this API pleasant to use" and "will this API help users solve the problems they have" and that's exactly why the coding agents did *so badly* at it.

This takes me back to the days I was obsessed with physics and, specifically, relativity. The laws of physics look simple and Newtonian in any small local area, but zoom out and spacetime curves in ways you can't predict from the local picture alone. Code is the same: at the level of a function or a class, there's usually a clear right answer, and AI is excellent there. But architecture is what happens when all those local pieces interact, and you can't get good global behaviour by stitching together locally correct components.

Knowing where you are on these axes at any given moment is, I think, the core skill of working with AI effectively.

vibe-coding month was the most extreme version of this. Yes, I understood the problem, but if I had been more disciplined about making hard design calls earlier, I could have converged on the right architecture much faster.

Tests created a similar false comfort[35]. Having 500+ tests felt reassuring, and AI made it easy to generate more. But neither humans nor AI are creative enough to foresee every edge case you'll hit in the future; there are several times in the vibe-coding phase where I'd come up with a test case and realise the design of some component was completely wrong and needed to be totally reworked. This was a significant contributor to my lack of trust and the decision to scrap everything and start from scratch.

Basically, I learned that the "normal rules" of software still apply in the AI age: if you don't have a fundamental foundation (clear architecture, well-defined boundaries) you'll be left eternally chasing bugs as they appear.

Something I kept coming back to was how little AI understood about the passage of time[36]. It sees a codebase in a certain state but doesn't *feel* time the way humans do. I can tell you what it feels like to use an API, how it evolved over months or years, why certain decisions were made and later reversed.

The natural problem from this lack of understanding is that you either make the same mistakes you made in the past and have to relearn the lessons *or* you fall into new traps which were successfully avoided the first time, slowing you down in the long run. In my opinion, this is a similar problem to why losing a high-quality senior engineer hurts a team so much: they carry history and context that doesn't exist anywhere else and act as a guide for others around them.

In theory, you can try to preserve this context by keeping specs and docs up to date. But there's a reason we didn't do this before AI: capturing implicit design decisions exhaustively is incredibly expensive and time-consuming to write down. AI can help draft these docs, but because there's no way to automatically verify that it accurately captured what matters, a human still has to manually audit the result. And that's still time-consuming.

There's also the context pollution problem. You never know when a design note about API A will echo in API B. Consistency is a huge part of what makes codebases work, and for that you don't just need context about what you're working on right now but also about other things which were designed in a similar way. Deciding what's relevant requires exactly the kind of judgement that institutional knowledge provides in the first place.

Subscribe to our newsletter

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 12 APR 2026 · SOURCE

For eight years, I've wanted a high-quality set of devtools for working with SQLite. Given how important SQLite is to the industry[1], I've long been puzzled that no one has invested in building a really good developer experience for it[2].

A couple of weeks ago, after ~250 hours of effort over three months[3] on evenings, weekends, and vacation days, I finally released syntaqlite (GitHub), fulfilling this long-held wish. And I believe the main reason this happened was because of AI coding agents[4].

Of course, there's no shortage of posts claiming that AI one-shot their project or pushing back and declaring that AI is all slop. I'm going to take a very different approach and, instead, systematically break down my experience building syntaqlite with AI, both where it helped *and* where it was detrimental.

I'll do this while contextualizing the project and my background so you can independently assess how generalizable this experience was. And whenever I make a claim, I'll try to back it up with evidence from my project journal, coding transcripts, or commit history[5].

Why I wanted it#

In my work on Perfetto, I maintain a SQLite-based language for querying performance traces called PerfettoSQL. It's basically the same as SQLite but with a few extensions to make the trace querying experience better. There are ~100K lines of PerfettoSQL internally in Google and it's used by a wide range of teams.

Having a language which gets traction means your users also start expecting things like formatters, linters, and editor extensions. I'd hoped that we could adapt some SQLite tools from open source but the more I looked into it, the more disappointed I was. What I found either wasn't reliable enough, fast enough[6], or flexible enough to adapt to PerfettoSQL. There was

clearly an opportunity to build something from scratch, but it was never the "most important thing we could work on". We've been reluctantly making do with the tools out there but always wishing for better.

On the other hand, there *was* the option to do something in my spare time. I had built lots of open source projects in my teens[7] but this had faded away during university when I felt that I just didn't have the motivation anymore. Being a maintainer is much more than just "throwing the code out there" and seeing what happens. It's triaging bugs, investigating crashes, writing documentation, building a community, and, most importantly, having a direction for the project.

But the itch of open source (specifically freedom to work on what I wanted while helping others) had never gone away. The SQLite devtools project was eternally in my mind as "something I'd like to work on". But there was another reason why I kept putting it off: it sits at the intersection of being both hard *and* tedious.

What makes it hard and tedious#

If I was going to invest my personal time working on this project, I didn't want to build something that only helped Perfetto: I wanted to make it work for *any* SQLite user out there[8]. And this means parsing SQL *exactly* like SQLite.

The heart of any language-oriented devtool is the parser. This is responsible for turning the source code into a "parse tree" which acts as the central data structure anything else is built on top of. If your parser isn't accurate, then your formatters and linters will inevitably inherit those inaccuracies; many of the tools I found suffered from having parsers which approximated the SQLite language rather than representing it precisely.

Unfortunately, unlike many other languages, SQLite has no formal specification describing how it should be parsed. It doesn't expose a stable API for its parser either. In fact, quite uniquely, in its implementation it doesn't even build a parse tree at all[9]! The only reasonable approach left in my opinion is to carefully extract the relevant parts of SQLite's source code and adapt it to build the parser I wanted[10].

This means getting into the weeds of SQLite source code, a fiendishly difficult codebase to understand. The whole project is written in C in an incredibly dense style; I've spent days just understanding the virtual table API[11] and implementation. Trying to grasp the full parser stack was daunting.

The tiredness feedback loop made it worse[29]. When I had energy, I could write precise, well-scoped prompts and be genuinely productive. But when I was tired, my prompts became vague, the output got worse, and I'd try again, getting more tired in the process. In these cases, AI was probably slower than just implementing something myself, but it was too hard to break out of the loop[30].

Losing touch#

Several times during the project, I lost my mental model of the codebase[31]. Not the overall architecture or how things fitted together. But the day-to-day details of what lived where, which functions called which, the small decisions that accumulate into a working system. When that happened, surprising issues would appear and I'd find myself at a total loss to understand what was going wrong. I hated that feeling.

The deeper problem was that losing touch created a communication breakdown[32]. When you don't have the mental thread of what's going on, it becomes impossible to communicate meaningfully with the agent. Every exchange gets longer and more verbose. Instead of "change FooClass to do X," you end up saying "change the thing which does Bar to do X". Then the agent has to figure out what Bar is, how that maps to FooClass, and sometimes it gets it wrong[33]. It's exactly the same complaint engineers have always had about managers who don't understand the code asking for fanciful or impossible things. Except now you've become that manager.

The fix was deliberate: I made it a habit to read through the code immediately after it was implemented and actively engage to see "how would I have done this differently?".

Of course, in some sense all of the above is also true of code I wrote a few months ago (hence the sentiment that AI code is legacy code), but AI makes the drift happen faster because you're not building the same muscle memory that comes from originally typing it out.

The slow corrosion#

There were some other problems I only discovered incrementally over the three months.

I found that AI made me procrastinate on key design decisions[34]. Because refactoring was cheap, I could always say "I'll deal with this later." And because AI could refactor at the same industrial scale it generated code, the cost of deferring felt low. But it wasn't: deferring decisions corroded my ability to think clearly because the codebase stayed confusing in the meantime. The

It was also invaluable for reacquainting myself with parts of the project I hadn't looked at for a few days[25]. I could control how deep to go: "tell me about this component" for a surface-level refresher, "give me a detailed linear walkthrough" for a deeper dive, "audit unsafe usages in this repo" to go hunting for problems. When you're context switching a lot, you lose context fast. AI let me reacquire it on demand.

More than I'd have built alone#

Beyond making the project exist at all, AI is also the reason it shipped as complete as it did. Every open source project has a long tail of features that are important but not critical: the things you know theoretically how to do but keep deprioritizing because the core work is more pressing. For syntaqlite, that list was long: editor extensions, Python bindings, a WASM playground, a docs site, packaging for multiple ecosystems[26]. AI made these cheap enough that skipping them felt like the wrong trade-off.

It also freed up mental energy for UX[27]. Instead of spending all my time on implementation, I could think about what a user's first experience should feel like: what error messages would actually help them fix their SQL, how the formatter output should look by default, whether the CLI flags were intuitive. These are the things that separate a tool people try once from one they keep using, and AI gave me the headroom to care about them. Without AI, I would have built something much smaller, probably no editor extensions or docs site. AI didn't just make the same project faster. It changed what the project *was*.

Where AI had its costs#

The addiction#

There's an uncomfortable parallel between using AI coding tools and playing slot machines[28]. You send a prompt, wait, and either get something great or something useless. I found myself up late at night wanting to do "just one more prompt," constantly trying AI just to see what would happen even when I knew it probably wouldn't work. The sunk cost fallacy kicked in too: I'd keep at it even in tasks it was clearly ill-suited for, telling myself "maybe if I phrase it differently this time."

There's also the fact that there are >400 rules in SQLite which capture the full surface area of its language. I'd have to specify in each of these "grammar rules" how that part of the syntax maps to the matching node in the parse tree. It's extremely repetitive work; each rule is similar to all the ones around it but also, by definition, different.

And it's not just the rules but also coming up with and writing tests to make sure it's correct, debugging if something is wrong, triaging and fixing the inevitable bugs people filed when I got something wrong…

For years, this was where the idea died. Too hard for a side project[12], too tedious to sustain motivation, too risky to invest months into something that might not work.

How it happened#

I've been using coding agents since early 2025 (Aider, Roo Code, then Claude Code since July) and they'd definitely been useful but never something I felt I could trust a serious project to. But towards the end of 2025, the models seemed to make a significant step forward in quality[13]. At the same time, I kept hitting problems in Perfetto which would have been trivially solved by having a reliable parser. Each workaround left the same thought in the back of my mind: maybe it's finally time to build it for real.

I got some space to think and reflect over Christmas and decided to really stress test the most maximalist version of AI: could I vibe-code the whole thing using just Claude Code on the Max plan (£200/month)?

Through most of January, I iterated, acting as semi-technical manager and delegating almost all the design and all the implementation to Claude. Functionally, I ended up in a reasonable place: a parser in C extracted from SQLite sources using a bunch of Python scripts, a formatter built on top, support for both the SQLite language and the PerfettoSQL extensions, all exposed in a web playground.

But when I reviewed the codebase in detail in late January, the downside was obvious: the codebase was complete spaghetti[14]. I didn't understand large parts of the Python source extraction pipeline, functions were scattered in random files without a clear shape, and a few files had grown to several thousand lines. It was *extremely* fragile; it solved the immediate problem *but* it was never going to cope with my larger vision, never mind integrating it into the Perfetto tools. The saving grace was that it had proved the approach was viable and generated more than 500 tests, many of which I felt I could reuse.

I decided to throw away everything and start from scratch while also switching most of the codebase to Rust[15]. I could see that C was going to make it difficult to build the higher level components like the validator and the language server implementation. And as a bonus, it would also let me use the same language for both the extraction and runtime instead of splitting it across C and Python.

More importantly, I completely changed my role in the project. I took ownership of all decisions[16] and used it more as "autocomplete on steroids" inside a much tighter process: opinionated design upfront, reviewing every change thoroughly, fixing problems eagerly as I spotted them, and investing in scaffolding (like linting, validation, and non-trivial testing[17]) to check AI output automatically.

The core features came together through February and the final stretch (upstream test validation, editor extensions, packaging, docs) led to a 0.1 launch in mid-March.

But in my opinion, this timeline is the least interesting part of this story. What I really want to talk about is what wouldn't have happened without AI and also the toll it took on me as I used it.

AI is why this project exists, and why it's as complete as it is#

Overcoming inertia#

I've written in the past about how one of my biggest weaknesses as a software engineer is my tendency to procrastinate when facing a big new project. Though I didn't realize it at the time, it could not have applied more perfectly to building syntaqlite.

AI basically let me put aside all my doubts on technical calls, my uncertainty of building the right thing and my reluctance to get started by giving me very concrete problems to work on. Instead of "I need to understand how SQLite's parsing works", it was "I need to get AI to suggest an approach for me so I can tear it up and build something better"[18]. I work so much better with concrete prototypes to play with and code to look at than endlessly thinking about designs in my head, and AI lets me get to that point at a pace I could not have dreamed about before. Once I took the first step, every step after that was so much easier.

AI turned out to be better than me at the act of writing code itself, assuming that code is obvious. If I can break a problem down to "write a function with this behaviour and parameters" or "write a class matching this interface," AI will build it faster than I would and, crucially, in a style that might well be more intuitive to a future reader. It documents things I'd skip, lays out code consistently with the rest of the project, and sticks to what you might call the "standard dialect" of whatever language you're working in[19].

That standardness is a double-edged sword. For the vast majority of code in any project, standard is exactly what you want: predictable, readable, unsurprising. But every project has pieces that are its edge, the parts where the value comes from doing something non-obvious. For syntaqlite, that was the extraction pipeline and the parser architecture. AI's instinct to normalize was actively harmful there, and those were the parts I had to design in depth and often resorted to just writing myself.

But here's the flip side: the same speed that makes AI great at obvious code also makes it great at refactoring. If you're using AI to generate code at industrial scale, you *have* to refactor constantly and continuously[20]. If you don't, things immediately get out of hand. This was the central lesson of the vibe-coding month: I didn't refactor enough, the codebase became something I couldn't reason about, and I had to throw it all away. In the rewrite, refactoring became the core of my workflow. After every large batch of generated code, I'd step back and ask "is this ugly?" Sometimes AI could clean it up. Other times there was a large-scale abstraction that AI couldn't see but I could; I'd give it the direction and let it execute[21]. If you have taste, the cost of a wrong approach drops dramatically because you can restructure quickly[22].

Teaching assistant#

Of all the ways I used AI, research had by far the highest ratio of value delivered to time spent.

I've worked with interpreters and parsers before but I had never heard of Wadler-Lindig pretty printing[23]. When I needed to build the formatter, AI gave me a concrete and actionable lesson from a point of view I could understand and pointed me to the papers to learn more. I could have found this myself eventually, but AI compressed what might have been a day or two of reading into a focused conversation where I could ask "but why does this work?" until I actually got it.

This extended to entire domains I'd never worked in. I have deep C++ and Android performance expertise but had barely touched Rust tooling or editor extension APIs. With AI, it wasn't a problem: the fundamentals are the same, the terminology is similar, and AI bridges the gap[24]. The VS Code extension would have taken me a day or two of learning the API before I could even start. With AI, I had a working extension within an hour.